

Code Explanation

BNCPLS IF23B 10K File Generation Code Explanation

The provided Python code is designed to generate 10K files as part of the BNCPLS IF23B process. It utilizes Apache Spark for data processing and involves several steps including data extraction, AURA payload processing, calculating 10K fields, determining file names, and writing the final files.

Overview of the Code

The code is structured around a main function that orchestrates the entire process. It starts by configuring logging, then proceeds to extract data from a source (simulated as Oracle database in the example), process AURA payloads, calculate necessary fields, determine file names, and finally write the 10K files. The process is wrapped in error handling to ensure robustness.

Step 1: Configuring Logging

The first step involves configuring the logging module to capture important events during the execution of the script. This includes setting the logging level, format, and output handler.

```
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.StreamHandler(sys.stdout)
    ]
)
```

Step 2: Extracting Source Data

This step is responsible for extracting data from the source database using a provided SQL query. In the example, the data extraction is simulated by creating a DataFrame with sample data.

```
def extract_source_data(spark: SparkSession, src_sql: str) -> DataFrame:
    # Create a schema that matches what we expect from the Oracle tables
    schema = StructType([
        StructField("ID", StringType(), False),
        StructField("POLICY_NUMBER", StringType(), False),
        # Other fields...
    ])

    # Create sample data
    sample_data = [
        ("ID001", "POL0000001", "M", "JOHN", "DOE", "2023-01-15", "PD94bwwgdmVyc2lrbj0iMS4wIiBlbmNvZGluZz0iVVRGLTgiPz48QXVyYT48RGF0YT5TYW1wbGUgQXVyYSBEYXRhPC9EYXRhPjwvQXVyYT4="),
        # Other sample records...
    ]

    df = spark.createDataFrame(sample_data, schema)
    return df
```

Step 3: Processing AURA Payloads

In this step, the code processes AURA payloads stored in the `I\_AURA\_INPUT\_BASE64` column of the DataFrame. It uses a user-defined function (UDF) `parse\_aura\_payload` to parse the payloads.

```
def process_aura_payloads(df: DataFrame, config: dict) -> DataFrame:
    # Register the UDF
    parse_aura_udf = udf(
        lambda i_aura_input_base64, policy_number: parse_aura_payload(
            i_aura_input_base64,
            policy_number,
            config['xslt_file'],
            config['aura15_xslt_file']
        ),
        StringType()
    )

    # Apply the UDF to process AURA payloads
    result_df = df.withColumn(
        "PARSED_AURA_OUTPUT",
        parse_aura_udf(col("I_AURA_INPUT_BASE64"), col("POLICY_NUMBER"))
    )

    return result_df
```

Step 4: Calculating 10K Fields and File Names

The code then calculates the necessary 10K fields using the `calculate\_10k\_fields` function and determines the file names using the `calculate\_filename` function.

```
df_with_fields = calculate_10k_fields(df_with_aura)
df_with_filename = calculate_filename(df_with_fields, config)
```

Step 5: Writing 10K Files

Finally, the processed data is written to files using the `write\_10k\_files` function.

```
write_10k_files(df_with_filename, config)
```

Step 6: Main Execution and Error Handling

The `main` function orchestrates the entire process and includes error handling to log any exceptions that occur during execution.

```
def main():
    try:
        # Extract source data
        source_df = extract_source_data(spark, config['src_sql'])

        # Process AURA payloads and subsequent steps...
```

```
        logging.info("BNCPLS IF23B 10K File Generation completed suc  
cessfully")  
    except Exception as e:  
        logging.error(f"Error in BNCPLS IF23B 10K File Generation: {  
str(e)}", exc_info=True)  
        raise  
    finally:  
        spark.stop()
```